

Preprocessing Implementation

Available Datasets (Artifacts)

The `data/processed/` directory contains the final optimized artifacts required for downstream pipelines (RFM analysis, clustering, and RecSys modeling):

- `train_processed.parquet`: Chronological training set (`t_dat < 2020-08-25`). Use for model training, interaction history, and RFM calculation.
 - `test_processed.parquet`: Chronological evaluation set (Final 28 days). Use strictly for RecSys validation and hold-out metrics.
 - `customers_processed.parquet`: Cleansed demographic and loyalty metadata. Use for user embeddings and clustering.
 - `articles_processed.parquet`: Master product catalog. Use for content-based feature extraction.
 - `id_mapping.json`: Translation dictionary bridging original Hex/String IDs to contiguous `Int32` integers. Use for model deployment or decoding predictions. But like, not actually needed for our project so just ignore this.
-

Pipeline Execution Summary

- Original Transactions: 31,788,324 rows
 - Final Transactions (Train + Test): 28,793,010 rows (~3M rows collapsed/purged)
 - Train Set Size: 27,803,789 interactions
 - Test Set Size: 989,221 interactions
-

Transformation Steps

1. Memory Optimization (ID Mapping)

- Action: Mapped `customer_id` (64-char hex) and `article_id` (int64) to contiguous zero-indexed `Int32`.
- Purpose: Drastically reduce RAM footprint during matrix operations and embedding layer initialization.
- Impact: Parquet sizes compressed significantly; RecSys sparse matrices can now be built natively without expensive string-hashing.

2. Transaction Deduplication

- Action: Grouped transactions by `[t_dat, customer_id, article_id, price, sales_channel_id]` and aggregated using a row count.
- Purpose: Resolve the 17.36% exact-duplicate anomaly caused by the lack of a baseline quantity feature.

- Impact: Exact duplicate rows dropped. A new `quantity` feature ensures interaction weight is preserved for models without leaking duplicate target rows.

3. Reseller / Outlier Purge

- Action: Filtered out customers with a total lifetime `quantity > 1000`.
- Purpose: Remove wholesale/reseller accounts that skew collaborative filtering algorithms via unnatural popularity bias.
- Impact: Surgically removed extreme outliers (the top ~20 accounts) while safely preserving legitimate high-engagement consumers (“whales” in the 150-600 item range).

4. RecSys Feature Engineering (Time Decay)

- Action: Engineered `days_ago` relative to the global max date, and computed `time_weight = exp(-0.023 * days_ago)`.
- Purpose: Provide a ready-to-use sequential weighting factor. A rate of `-0.023` assigns a 30-day half-life to past purchases.
- Impact: Enables downstream models to prioritize “fresh” interactions natively, directly addressing the 90-day SKU shelf life discovered in EDA.

5. Customer Profiling & Masking

- Action:
 - Binary Flags: Imputed `FN` and `Active` nulls with `0` (Int8).
 - Geo-Masking: Overwrote the 626k-transaction anomaly code (`2c29ae65...`) to `Unknown`.
 - Affinitive Age Imputation: Imputed missing age using the median age of the customer’s most purchased product index (e.g., `Divided` → 29, `Ladieswear` → 33).
- Purpose: Ensure a completely dense user-feature matrix without introducing fatal statistical flaws (e.g., destroying bimodal distributions via global median imputation).
- Impact: Clustering algorithms (e.g., K-Means) will no longer group missing-age users into a fake, artificial “middle-age” valley.

6. Temporal Train/Test Split

- Action: Partitioned the timeline using the global max date minus 28 days.
- Purpose: Prevent data leakage (future predicting past) and ensure the test set represents a realistic production deployment scenario.
- Impact: Yielded a robust ~1M row evaluation set, providing sufficient density to evaluate cold-start and RecSys metrics accurately.